| Course Code: CS1103<br>Name of the Programme: B.Sc. Computer Science (Hons.)<br>Name of the Course: Operating Systems<br>Semester: II | | |
|---|---|---|
| **Course credit** | **No. of hours per week (L + T + P)** | **Total no. of Teaching hours** |
| 3 | 2+0+2 | 60 |

**Course Objectives**:

a) To equip students with a strong foundational knowledge of OS concepts.

b) To covers key components of an operating system including, Process management Memory management, Deadlock detection and handling, Device and I/O management.

c) To introduce students about Advanced optimization techniques, Scheduling algorithms and mechanisms, System-level resource management.

d) To emphasize both theoretical understanding and practical application.

| | **6** |
|---|---|

**Basics of operating systems, Process management**

Generations, Types, Structure, Services, System Boot, System Programs, Protection and Security.

Process Concepts, Process States, Process Control Block, Scheduling-Criteria, Scheduling Algorithms and their Evaluation.

| **Module - 2** | **No. of hours** |
|---|---|
| | **6** |

**Thread API**

Thread Creation, Completion, Locks, Condition Variables, Compilation. Inter Process Communication and Synchronization: Software Approaches, Principles of Concurrency, Hardware Support, Semaphores, Message Passing,

| **Module 3** | **6** |
|---|---|

**System Calls, Traps, Device Drivers, and Deadlocks**

System Calls – Concepts and Execution Flow, User Mode and Kernel Mode, Trap Handling and Exception Flow, Interrupt Service Routines (ISRs), Implementation of System Calls in xv6, Device Communication and Drivers - Deadlocks – Characterization, Deadlocks in I/O and Device Handling, Prevention, Detection, and Avoidance Techniques in Kernel-Level Resource Management.

| **Module - 4** | **No. of hours** |
|---|---|
| | **6** |

**Memory Management**

Main Memory, Swapping, Contiguous Memory Allocation, Paging, Structure of Page Table, Segmentation, Virtual Memory, Demand Paging, Page Replacement Algorithms, Allocation of Frames, Thrashing.

| **Module 5** | **6** |
|---|---|

**Storage Structure**

Overview of Mass Storage Structure, Disk Structure, Disk Scheduling, Disk Management, Swap-Space Management, I/O System Overview, I/O Hardware, Application I/O Interface.

| Course outcomes |
|---|
| a) Understanding of the historical evolution and basic principles of operating systems. |
| b) Understand the process management techniques and scheduling. |
| c) Analyze deadlock conditions and propose methods to avoid or resolve them. |
| d) Analyze and address issues such as memory fragmentation and dynamic memory allocation. |

| Text Books | |
|---|---|
| 1 | Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson Education. ISBN 978-9352866717 |
| 2 | Silberschatz, A., Galvin, P. B., & Gagne, G. (2010). *Operating system concepts essentials*. John Wiley & Sons. ISBN: 978-0470889206 |
| 3 | Cox, R., Kaashoek, F., & Morris, R. (2022). *xv6: A simple, Unix-like teaching operating system* (Rev. 3, RISC-V ed.). MIT CSAIL. |

| Reference books | |
|---|---|
| 1 | Silberschatz, A., Galvin, P. B., & Gagne, G. (2012). *Operating system concepts* (9th ed.). Wiley. ISBN: 978-1118063330 |
| 2 | Tanenbaum, A. S., & Bos, H. (2023). *Modern operating systems* (Global 5th ed.). Pearson. ISBN: 978-1292459660 |

| **Lab Programs** | **[30 Hrs]** |
|---|---|
| **1** | Process Creation in Unix Shell: Creating Child Processes and Displaying Parent-Child Relationships Using ps. |
| **2** | Process Control: Create a shell script that demonstrates process control commands like bg, fg, jobs, and kill. Implement a simple background task and manage its execution. |
| **3** | Basic Shell Script for Job Scheduling: Develop a shell script that takes a list of commands and executes them with a specified delay using the sleep command. Demonstrate job scheduling in the shell. |
| **4** | File Operations: Create a shell script that performs file operations, such as creating, copying, moving, and deleting files using commands like touch, cp, mv, and rm. Include error handling to manage file operations gracefully. |
| **5** | File Permissions: Write a shell script to modify file permissions and ownership using chmod and chown. Include user input for the filename and desired permissions and display the new file permissions. |
| **6** | Directory Management: Develop a shell script that creates, lists, and deletes directories. Use commands like mkdir, ls, and rmdir, and include user prompts for directory names. |
| **7** | Dynamic Memory Check: Write a shell script that monitors memory usage on the system using commands like free and top. Display the current memory usage and available memory. |
| **8** | Environment Variables: Create a shell script that sets, exports, and displays environment variables. Use printenv and env to show the current environment settings. |
| **9** | I/O Redirection in Unix Shell: Manipulating Data Streams with Enhanced File Operations. |
| **10** | Command Piping in Unix Shell: Exploring Operating System Features. |

| | | |
|---|---|---|
| **Name of the Programme:** | **B.Sc. Computer Science (Hons.)** | |
| **Title of the Course:** | **Operating Systems** | |
| **Course code** | **CS1103** | |
| **Semester:** | **II** | |
| **Names of the Course Instructor(s)** | **Prof . Aruna S**<br>**Prof Anoop A** | |
| **Course credit** | **No. of hours per week (L + T + P)** | **Total no. of Teaching hours** |
| **3** | **2+0+2** | **60** |

| Session | Module | Topic | Pedagogy / Activities | Reference |
|---|---|---|---|---|
| 1 | 1 | Theory: OS Basics, Structure, Types | Lecture | Stallings Ch.1 |
| 2 | 1 | Theory: System Calls, Booting, OS Services | Demo | Silberschatz Ch.2 |
| 3 | 1 | Lab Program 1: Process Creation using fork() – Part 1 | Implementation & testing | Lab 1 |
| 4 | 1 | Lab Program 1: Process Creation using fork() – Part 2 | Demonstrate parent-child outputs | Lab 1 |
| 5 | 1 | Theory: Processes & PCB | Diagrams | Silberschatz Ch.3 |
| 6 | 1 | Theory: Process Scheduling | Gantt charts | Silberschatz Ch.5 |
| 7 | 1 | Lab Program 2: Orphan & Zombie Processes – Part 1 | Zombie example | Lab 2 |
| 8 | 1 | Lab Program 2: Orphan & Zombie Processes – Part 2 | Orphan example | Lab 2 |
| 9 | 1 | Theory: Interprocess Communication | Board diagrams | Silberschatz Ch.3 |
| 10 | 1 | Theory: IPC Mechanisms (Overview) | Discussion | Silberschatz Ch.3 |
| 11 | 1 | Lab Program 3: IPC using Pipes – Part 1 | Read/write demo | Lab 3 |
| 12 | 1 | Lab Program 3: IPC using Pipes – Part 2 | Two-way communication | Lab 3 |
| | | | | |
| 13 | 2 | Theory: Critical Section Problem | Race condition demo | Silberschatz Ch.6 |
| 14 | 2 | Theory: Semaphores & Mutex | Animated explanation | Stalllings Ch.5 |

| | | | | |
|---|---|---|---|---|
| 15 | 2 | Lab Program 4: Message Queue IPC – Part 1 | Send/receive msgs | Lab 4 |
| 16 | 2 | Lab Program 4: Message Queue IPC – Part 2 | Multi-process messaging | Lab 4 |
| 17 | 2 | Theory: Deadlocks & RAG | Examples | Russ Cox Ch.6 |
| 18 | 2 | Theory: Deadlock Handling Techniques | Case discussion | Russ Cox Ch.6 |
| 19 | 2 | Lab Program 5: Shared Memory – Part 1 | shmget, shmat tests | Lab 5 |
| 20 | 2 | Lab Program 5: Shared Memory – Part 2 | Data writing/reading | Lab 5 |
| 21 | 2 | Theory: Producer–Consumer Problem | Concept explanation | Silberschatz Ch.6 |
| 22 | 2 | Theory: Reader–Writer Problem | Board explanation | Silberschatz Ch.6 |
| 23 | 2 | Lab Program 6: Producer–Consumer (Semaphores) | wait/signal | Lab 6 |
| 24 | 2 | Lab Program 7: Reader–Writer Problem | Reader & writer priority | Lab 7 |
| | | | | |
| 25 | 3 | Theory: Memory Basics, Binding | Diagram | Silberschatz Ch.8 |
| 26 | 3 | Theory: MFT/MVT | Fragmentation demo | Silberschatz Ch.7 |
| 27 | 3 | Lab Program 8: Dining Philosophers – Part 1 | Deadlock demo | Lab 8 |
| 28 | 3 | Lab Program 8: Dining Philosophers – Part 2 | Avoid starvation | Lab 8 |
| 29 | 3 | Theory: Paging, TLB | Visual flow | Stallings Ch.7 |
| 30 | 3 | Theory: Segmentation | Diagram | Silberschatz Ch.7 |
| 31 | 3 | Lab Program 9: File Allocation – Part 1 | Structures | Lab 9 |
| 32 | 3 | Lab Program 9: File Allocation – Part 2 | Linked tables | Lab 9 |
| 33 | 3 | Theory: Page Replacement (FIFO/LRU/OPT) | Examples | Silberschatz Ch.9 |
| 34 | 3 | Theory: Allocation Methods | First/Best/Worst fit | Silberschatz Ch.8 |
| 35 | 3 | Lab Program 9: File Allocation – Part 3 | Indexed method | Lab 9 |
| 36 | 3 | Lab Program 10: Disk Scheduling – FCFS | Implementation | Lab 10 |
| | | | | |
| 37 | 4 | Theory: File Concepts | File structure demo | Russ Cox Ch.8 |

| | | | | |
|---|---|---|---|---|
| 38 | 4 | Theory: Directory Structure | FS tree explanation | Silberschatz Ch.10 |
| 39 | 4 | Lab Program 10: Disk Scheduling – SSTF | Implementation | Lab 10 |
| 40 | 4 | Lab Program 10: Disk Scheduling – SCAN/C-SCAN | Implementation | Lab 10 |
| 41 | 4 | Theory: Protection & Access Rights | chmod examples | Silberschatz Ch.11 |
| 42 | 4 | Theory: File Allocation Methods | Comparison | Silberschatz Ch.12 |
| 43 | 4 | Lab Program 9: Contiguous Allocation | Simulation | Lab 9 |
| 44 | 4 | Lab Program 9: Linked Allocation | Implementation | Lab 9 |
| 45 | 4 | Theory: Free Space Management | Bitmap/free list | Silberschatz Ch.11 |
| 46 | 4 | Theory: FS Implementation | Layout diagrams | Silberschatz Ch.12 |
| 47 | 4 | Lab Program 9: Indexed Allocation | Index block creation | Lab 9 |
| 48 | 4 | Lab Program 10: Disk Scheduling (Simulation) | Simulation | Lab 10 |
| | | | | |
| 49 | 5 | Theory: Mass Storage Systems | HDD/SSD discussion | Silberschatz Ch.12 |
| 50 | 5 | Theory: Disk Structure & Formatting | Example view | Silberschatz Ch.12 |
| 51 | 5 | Lab Program 10: Disk Scheduling – FCFS | Implementation | Lab 10 |
| 52 | 5 | Lab Program 10: Disk Scheduling – SSTF | Implementation | Lab 10 |
| 53 | 5 | Theory: Disk Scheduling | FCFS, SSTF, SCAN | Silberschatz Ch.12 |
| 54 | 5 | Theory: I/O System Architecture | Interrupts, DMA | Silberschatz Ch.13 |
| 55 | 5 | Lab Program 10: Disk Scheduling – SCAN/C-SCAN | Implementation | Lab 10 |
| 56 | 5 | Lab Program 1 (Revision): fork() Mechanism | Reinforcement | Lab 1 |
| 57 | 5 | Theory: I/O Interface | Streams, buffering | Silberschatz Ch.13 |
| 58 | 5 | Theory: Swap Management | Swap space structure | Silberschatz Ch.8 |
| 59 | 5 | Lab Program 3 (Revision): Pipe IPC | Reinforcement | Lab 3 |
| 60 | 5 | Lab Program 5 (Revision): Shared Memory | Reinforcement | Lab 5 |

| CIE–1 (20 marks) — *Two Components* | |
|---|---|
| **Component** | **Marks** |

| | |
|---|---|
| **1. Quiz** | **15 marks** |
| **Assignment** | **5 marks** |
| CIE–2 (25 marks) | |
| **Component** | **Marks** |
| **1. Mid–Semester Theory Examination** | **25 marks** |
| CIE–3 (25 marks) — *Two Components* | |
| **Component** | **Marks** |
| **Lab test(20) + Lab Program Submission(5)** | **25 marks** |

CIE–1 – Rubrics for Evaluation (Total Marks: 20)

| CIE-1 Components | Rubrics for Assessment |
|---|---|
| **CIE-1 – Quiz (Graded Component 1)** | Quiz will be conducted for **15 marks**. Each correct answer carries **1 mark**. **No negative marking** for wrong answers. Questions will cover **core Operating System concepts**, including OS structure, system calls, process management, CPU scheduling, synchronization, memory management, file systems, and I/O systems. |
| **Assignment – Graded Component 2** | Assignment carries **5 marks**. Students must complete the assigned **OS analytical/problem-solving task** (numericals, case study, or concept explanation) and submit it within the deadline through the designated LMS/Google Classroom. *(Refer Table-1 for detailed rubric.)* |

Table-1 – Graded Component-2 (5 Marks) – OS Assignment Evaluation

| Criteria | Excellent (5 Marks) | Satisfactory (3 Marks) | Needs Improvement (0 Marks) |
|---|---|---|---|
| **Understanding & Correctness** | Demonstrates **clear conceptual understanding** of OS topics; solutions are **correct, complete, and well-reasoned**. | Shows partial understanding; minor conceptual or calculation errors present. | Displays poor understanding or incorrect/incomplete solutions. |

| | | | |
|---|---|---|---|
| **Presentation & Clarity** | Answers are **well-structured**, neat, and logically explained using diagrams/examples where applicable. | Adequate presentation but lacks clarity or proper explanation. | Poor presentation, unclear or unorganized responses. |
| **Timely Submission** | Submitted on or before the deadline. | Minor delay or formatting issues. | Late submission or not submitted. |

CIE–2 (25 Marks)

| Component | Marks |
|---|---|
| **Mid-Semester Theory Examination** | **25 Marks** |

CIE–3 (25 Marks) — Practical Component (Operating Systems)

| \Component | Marks |
|---|---|
| **Lab Test+viva** | 16+4 Marks *(Refer Table-3)* |
| **Lab Program Submission** | 5 Marks*(Refer Table-4)* |
| **Total** | **25 Marks** |

CIE-3 – Rubrics for Evaluation (Total Marks: 25)

# Table–3: Lab Test Rubrics (Total: 20 Marks)

**Lab Test Total = 16 (Execution) + 4 (Viva) = 20 Marks**

## A. Lab Execution Rubrics (Max: 16 Marks)

| Sl. No | Criteria | Measuring Method | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Poor (0–1 Marks) |
|---|---|---|---|---|---|---|
| 1 | Understanding of OS Problem Statement | Observation | Thorough understanding of OS concepts (process creation, IPC, synchronization, memory management) with correct approach. | Good understanding with minor gaps. | Basic understanding with limited clarity. | Incorrect or poor understanding. |
| 2 | Program Execution | Observation | Correct and optimized execution of OS programs (fork, pipes, shared memory, scheduling, synchronization). | Executes with minor errors but correct output. | Executes with noticeable errors. | Fails to execute correctly. |
| 3 | Correctness of Output | Observation | Output correct for all test cases. | Output correct for most cases. | Partially correct output. | Incorrect output. |
| 4 | Use of OS Concepts | Observation | Appropriate and effective use of OS concepts and system calls. | Uses concepts with minor issues. | Limited use of concepts. | Incorrect use of concepts. |

## B. Viva Voce Rubrics (Max: 4 Marks)

| Sl. No | Criteria | Measuring Method | Excellent (2 Marks) | Good (1 Mark) | Poor (0–0.5 Marks) |
|---|---|---|---|---|---|
| 1 | Conceptual Understanding of OS Techniques | Viva Voce | Clearly explains OS concepts, algorithms, system calls, and program logic. | Explains concepts with minor gaps. | Unable to explain concepts or code. |
| 2 | Application of OS Concepts | Viva Voce | Justifies design choices (IPC method, synchronization primitive, scheduling logic). | Partial justification with gaps. | Unable to relate concepts to implementation. |

**Table–4: Lab Program Submission Rubrics (Total: 5 Marks)**

| Criteria | Excellent (5 Marks) | Satisfactory (3 Marks) | Needs Improvement (0–2 Marks) |
|---|---|---|---|
| **Completeness of Lab Programs(2)** | All prescribed OS lab programs completed and submitted on time. | Most programs complted with minor omissions. | Several programs missing or incomplete. |
| **Correctness & Output(2)** | Programs execute correctly with expected outputs. | Minor execution errors. | Incorrect or failed execution. |
| **Documentation(1)** | Well-documented code with comments, screenshots, and clear explanations. | Adequate documentation with limited clarity. | Poor or missing documentation. |